```
            ##### Team IOTA ######

#importing libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import warnings
warnings.filterwarnings('ignore')
```

```
from google.colab import files

# Choose your file from our device to import
uploaded = files.upload()
```

Choose Files Crop and fe… dataset.csv

Crop and fertilizer dataset.csv(text/csv) - 376974 bytes, last modified: 9/26/2025 - 100% done

```
Saving Crop and fertilizer dataset.csv to Crop and fertilizer dataset.csv
```

```
#importing the dataset
data = pd.read_csv('Crop and fertilizer dataset.csv')
data.head()
```

	District_Name	Soil_color	Nitrogen	Phosphorus	Potassium	pH	Rainfall	Temperature	Crop	Fertilizer	
0	Kolhapur	Black	75	50	100	6.5	1000	20	Sugarcane	Urea	https://youtu.be/2t5Am
1	Kolhapur	Black	80	50	100	6.5	1000	20	Sugarcane	Urea	https://youtu.be/2t5Am
2	Kolhapur	Black	85	50	100	6.5	1000	20	Sugarcane	Urea	https://youtu.be/2t5Am
3	Kolhapur	Black	90	50	100	6.5	1000	20	Sugarcane	Urea	https://youtu.be/2t5Am
4	Kolhapur	Black	95	50	100	6.5	1000	20	Sugarcane	Urea	https://youtu.be/2t5Am

Next steps: Generate code with `data` New interactive sheet

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4513 entries, 0 to 4512
Data columns (total 11 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   District_Name  4513 non-null   object
 1   Soil_color     4513 non-null   object
 2   Nitrogen       4513 non-null   int64
 3   Phosphorus     4513 non-null   int64
 4   Potassium      4513 non-null   int64
 5   pH             4513 non-null   float64
 6   Rainfall       4513 non-null   int64
 7   Temperature    4513 non-null   int64
 8   Crop           4513 non-null   object
 9   Fertilizer     4513 non-null   object
 10  Link           4513 non-null   object
dtypes: float64(1), int64(5), object(5)
memory usage: 388.0+ KB
```

```
# Clean column names in one step
data.rename(columns={
    'District_Name': 'District',
    'Soil_color': 'Soil_Color',
    'Nitrogen': 'N',
    'Phosphorus': 'P',
    'Potassium': 'K',
    'pH': 'pH',
    'Rainfall': 'Rainfall',
    'Temperature': 'Temp',
    'Crop': 'Crop',
    'Fertilizer': 'Fertilizer',
    'Link': 'Reference_Link'
}, inplace=True)
```

```
#checking unique values
data.nunique()
```

	0
District	5
Soil_Color	7
N	27
P	17
K	30
pH	7
Rainfall	15
Temp	7
Crop	16
Fertilizer	19
Reference_Link	278

dtype: int64

```
#checking for null values
data.isna().sum()
```

	0
District	0
Soil_Color	0
N	0
P	0
K	0
pH	0
Rainfall	0
Temp	0
Crop	0
Fertilizer	0
Reference_Link	0

dtype: int64

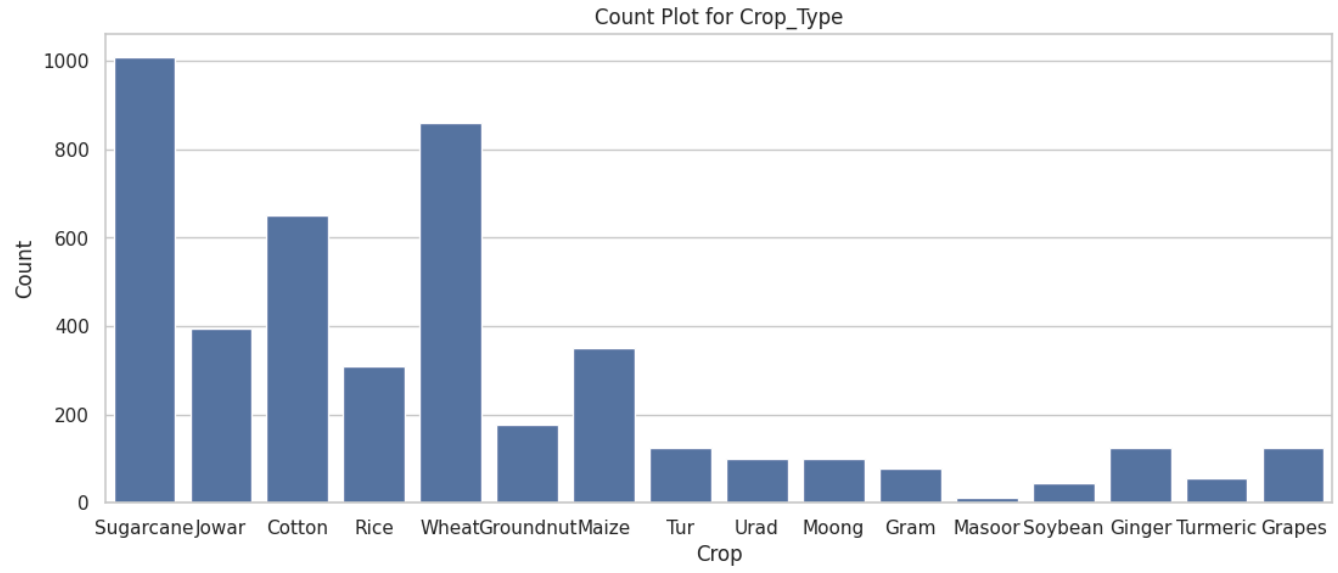
```
data['Fertilizer'].unique()
```

```
array(['Urea', 'DAP', 'MOP', '10:26:26 NPK', 'SSP', 'Magnesium Sulphate',
       '13:32:26 NPK', '12:32:16 NPK', '50:26:26 NPK', '19:19:19 NPK',
       'Chilated Micronutrient', '18:46:00 NPK', 'Sulphur',
       '20:20:20 NPK', 'Ammonium Sulphate', 'Ferrous Sulphate',
       'White Potash', '10:10:10 NPK', 'Hydrated Lime'], dtype=object)
```

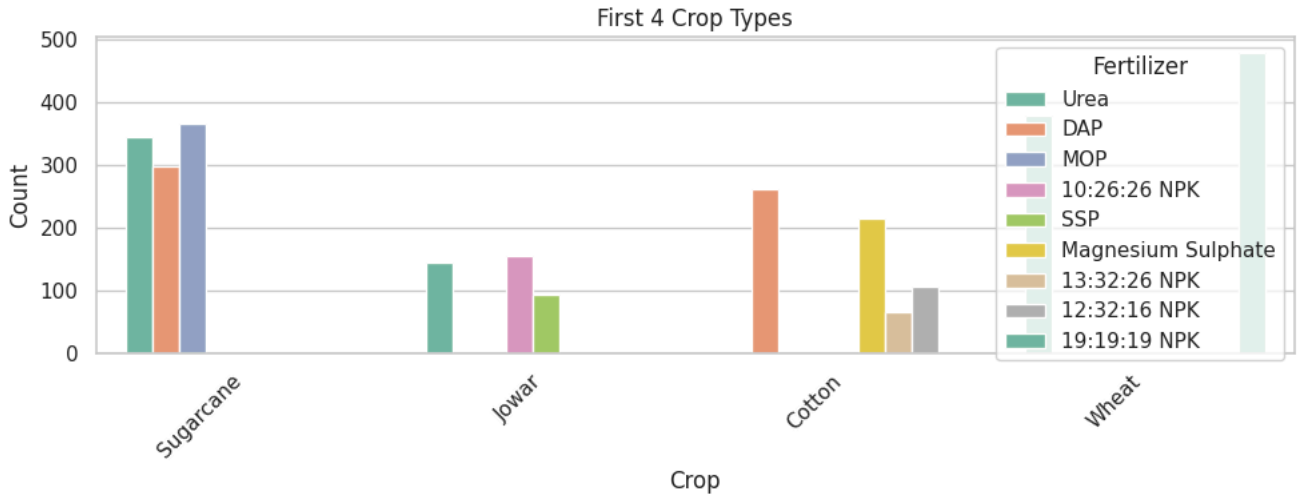
```
#statistical parameters
data.describe(include='all')
```

	District	Soil_Color	N	P	K	pH	Rainfall	Temp	Crop	Fertilizer	
count	4513	4513	4513.000000	4513.000000	4513.000000	4513.000000	4513.000000	4513.000000	4513	4513	
unique	5	7	NaN	NaN	NaN	NaN	NaN	NaN	16	19	
top	Kolhapur	Black	NaN	NaN	NaN	NaN	NaN	NaN	Sugarcane	Urea	http
freq	1430	2260	NaN	NaN	NaN	NaN	NaN	NaN	1010	1364	
mean	NaN	NaN	95.409927	54.341901	63.595170	6.715267	819.189010	25.915134	NaN	NaN	
std	NaN	NaN	38.060648	16.551991	35.691911	0.625198	251.730813	5.897328	NaN	NaN	
min	NaN	NaN	20.000000	10.000000	5.000000	5.500000	300.000000	10.000000	NaN	NaN	
25%	NaN	NaN	60.000000	40.000000	40.000000	6.000000	600.000000	20.000000	NaN	NaN	
50%	NaN	NaN	105.000000	55.000000	55.000000	6.500000	800.000000	25.000000	NaN	NaN	
75%	NaN	NaN	125.000000	65.000000	75.000000	7.000000	1000.000000	30.000000	NaN	NaN	
max	NaN	NaN	150.000000	90.000000	150.000000	8.500000	1700.000000	40.000000	NaN	NaN	

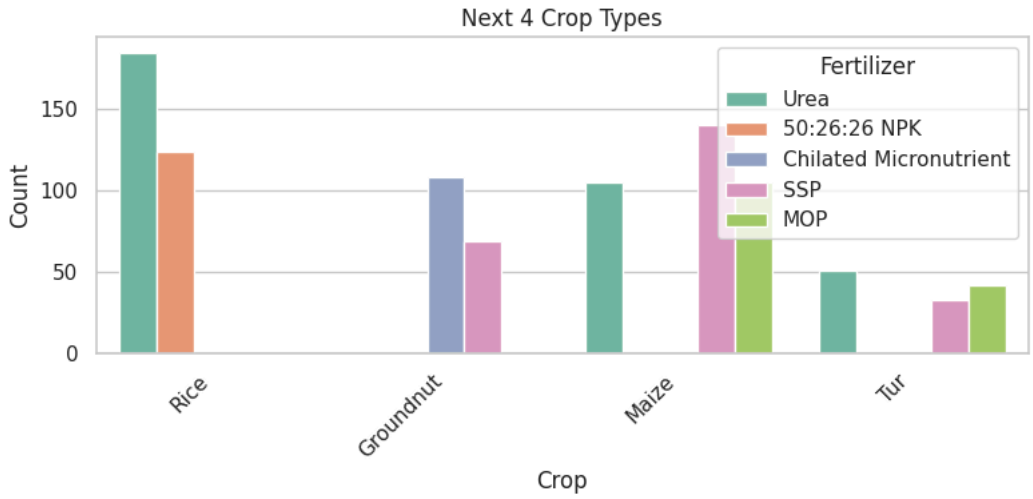
```
plt.figure(figsize=(13, 5))
sns.set(style="whitegrid")
sns.countplot(data=data, x='Crop')
plt.title('Count Plot for Crop_Type')
plt.xlabel('Crop')
plt.ylabel('Count')
plt.show()
```




```
#first 4 crop types
part1_data = data[data['Crop'].isin(data['Crop'].value_counts().index[:4])]

# Create the first countplot
plt.figure(figsize=(10, 4))
sns.set(style="whitegrid")
sns.countplot(data=part1_data, x='Crop', hue='Fertilizer', width=0.8, palette='Set2')
plt.title('First 4 Crop Types')
plt.xlabel('Crop')
plt.ylabel('Count')
plt.legend(title='Fertilizer')
plt.xticks(rotation=45, horizontalalignment='right')
plt.tight_layout()
plt.show()
```



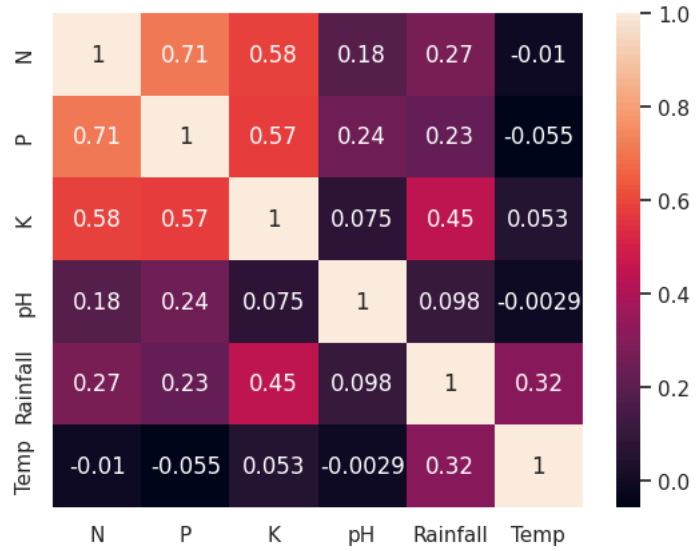

```
# Split the data into three parts: next 4 crop types
part2_data = data[data['Crop'].isin(data['Crop'].value_counts().index[4:8])]

# Create the second countplot
plt.figure(figsize=(8, 4))
sns.set(style="whitegrid")
sns.countplot(data=part2_data, x='Crop', hue='Fertilizer', width=0.8, palette='Set2')
plt.title('Next 4 Crop Types')
plt.xlabel('Crop')
plt.ylabel('Count')
plt.legend(title='Fertilizer')
plt.xticks(rotation=45, horizontalalignment='right')
plt.tight_layout()
plt.show()
```




```
#  this plot is provides insights into how different crop types are distributed based on the type of fertilizer used.
# The x-axis represents the different crop types.
# The y-axis represents the count (the number of occurrences) of each crop type in the dataset.
```

```
sns.heatmap(data.select_dtypes(include=[np.number]).corr(), annot=True)
```

```
<Axes: >
```




```
#there is no strong correlation between any input attributes
```

```
#encoding the labels for categorical variables
from sklearn.preprocessing import LabelEncoder
#transform non-numeric data into a numeric format
```

```
#encoding Soil Type variable
encode_District = LabelEncoder()

#fitting the label encoder
data.District_Type = encode_District.fit_transform(data.District)

#creating the DataFrame
District = pd.DataFrame(zip(encode_District.classes_,encode_District.transform(encode_District.classes_)),columns=['Original','Enc
District = District.set_index('Original')
District
```

Original	Encoded
Kolhapur	0
Pune	1
Sangli	2
Satara	3
Solapur	4

Next steps: Generate code with `District` New interactive sheet

```
#encoding Soil Type variable
encode_soil = LabelEncoder()

#fitting the label encoder
data.Soil_Color = encode_soil.fit_transform(data.Soil_Color)

#creating the DataFrame
Soil_Color = pd.DataFrame(zip(encode_soil.classes_,encode_soil.transform(encode_soil.classes_)),columns=['Original','Encoded'])
Soil_Color = Soil_Color.set_index('Original')
Soil_Color
```

Original	Encoded
0	0
1	1
2	2
3	3
4	4
5	5
6	6

Next steps: Generate code with Soil_Color New interactive sheet

```
#encoding Crop Type variable
encode_crop = LabelEncoder()

#fitting the label encoder
data.Crop = encode_crop.fit_transform(data.Crop)

#creating the DataFrame
Crop = pd.DataFrame(zip(encode_crop.classes_,encode_crop.transform(encode_crop.classes_)),columns=['Original','Encoded'])
Crop = Crop.set_index('Original')
Crop
```

Original	Encoded
Cotton	0
Ginger	1
Gram	2
Grapes	3
Groundnut	4
Jowar	5
Maize	6
Masoor	7
Moong	8
Rice	9
Soybean	10
Sugarcane	11
Tur	12
Turmeric	13
Urad	14
Wheat	15

Next steps: Generate code with Crop New interactive sheet

```
#encoding Fertilizer variable
encode_fert = LabelEncoder()

#fitting the label encoder
data.Fertilizer = encode_fert.fit_transform(data.Fertilizer)

#creating the DataFrame
Fertilizer = pd.DataFrame(zip(encode_fert.classes_,encode_fert.transform(encode_fert.classes_)),columns=['Original','Encoded'])
Fertilizer = Fertilizer.set_index('Original')
Fertilizer
```

Original	Encoded
10:10:10 NPK	0
10:26:26 NPK	1
12:32:16 NPK	2
13:32:26 NPK	3
18:46:00 NPK	4
19:19:19 NPK	5
20:20:20 NPK	6
50:26:26 NPK	7
Ammonium Sulphate	8
Chilated Micronutrient	9
DAP	10
Ferrous Sulphate	11
Hydrated Lime	12
MOP	13
Magnesium Sulphate	14
SSP	15
Sulphur	16
Urea	17
White Potash	18

Next steps: Generate code with Fertilizer | New interactive sheet

```
from sklearn.preprocessing import LabelEncoder
encode_link = LabelEncoder()
data['Link'] = encode_link.fit_transform(data['Link'])
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3804         try:
-> 3805             return self._engine.get_loc(casted_key)
   3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'Link'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
                                          2 frames
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3810             ):
   3811                 raise InvalidIndexError(key)
-> 3812             raise KeyError(key) from err
   3813         except TypeError:
   3814             # If we have a listlike key, _check_indexing_error will raise

KeyError: 'Link'
```

Next steps: Explain error

```
#changing the column names
data.rename(columns={'Link' : 'Link'
                     },
            inplace=True)
```

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd

# Initialize encoder
encode_link = LabelEncoder()

# Fit + transform the correct column
data['Reference_Link'] = encode_link.fit_transform(data['Reference_Link'])

# Create a mapping DataFrame
Link = pd.DataFrame({
    'Original': encode_link.classes_,
    'Encoded': encode_link.transform(encode_link.classes_)
}).set_index('Original')

print(Link.head())
```

```
                                                Encoded
Original
https://www.youtube.com/live/vX-DfTU-KbE?featur...        0
https://youtu.be/2t5Am0xLTOo                              1
https://youtu.be/EhyW9blAneM                              2
https://youtu.be/RcLaiAgFzrQ                              3
https://youtu.be/RiBSAjxgFvU                              4
```

```
print(data.columns.tolist())
```

```
['District', 'Soil_Color', 'N', 'P', 'K', 'pH', 'Rainfall', 'Temp', 'Crop', 'Fertilizer', 'Reference_Link']
```

```
# X = all columns except Fertilizer and Reference_Link
X = data.drop(['Fertilizer', 'Reference_Link'], axis=1)

# y = both Fertilizer and Reference_Link
y = data[['Fertilizer', 'Reference_Link']]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (4513, 9)
y shape: (4513, 2)
```

```
#splitting the data into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
from sklearn.preprocessing import LabelEncoder

# Apply LabelEncoder to each categorical column
label_encoders = {}
for col in X.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])
    label_encoders[col] = le  # store encoder for inverse transform later
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
import pandas as pd

# Step 1: Split into train/test before encoding
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 2: Encode categorical columns
label_encoders = {}
for col in X_train.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    X_train[col] = le.fit_transform(X_train[col])
    X_test[col] = le.transform(X_test[col])  # use same mapping
```

```
    label_encoders[col] = le

# Step 3: Scale numerical features
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
```

```
X_train shape: (3610, 9)
X_test shape: (903, 9)
```

```
#Feature Scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
#Building the model
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

Gemini

```
#predicting the test set results
y_pred = rf.predict(X_test)
```

```
# Features = all except Fertilizer (target) and Reference_Link (not needed)
X = data.drop(['Fertilizer', 'Reference_Link'], axis=1)

# Target = only Fertilizer
y = data['Fertilizer']
```

```
# y is only Fertilizer now
print(type(y))          # should be <class 'pandas.core.series.Series'>
print(y.shape)          # number of rows
print(y.head())         # first 5 values
print(y.value_counts()) # count of each class
```

```
<class 'pandas.core.series.Series'>
(4513,)
0    17
1    17
2    17
3    17
4    17
Name: Fertilizer, dtype: int64
Fertilizer
17    1364
10     667
13     571
5      480
15     417
14     215
1      156
7      124
9      108
2      106
11      68
3       66
8       50
0       50
12      25
18      19
6       15
4        6
16       6
Name: count, dtype: int64
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
print(y_train.head())
```

```
y_train shape: (3610,)
y_test shape: (903,)
3903    11
999     17
2366    17
1474    10
3859    17
Name: Fertilizer, dtype: int64
```

```
print(y_test.shape, y_pred.shape)
print(y_test[:5])
print(y_pred[:5])
```

```
(903,) (903, 2)
4145     5
544     10
157     10
274     14
538      2
Name: Fertilizer, dtype: int64
[[ 5  5]
 [10  0]
 [10  0]
 [14  0]
 [ 2  0]]
```

```
import numpy as np

# Reduce predictions (903,2) → (903,)
y_pred = np.argmax(y_pred, axis=1)

print(y_pred.shape)  # (903,)
print(y_pred[:10])   # first 10 predicted classes
```

```
(903,)
[0 0 0 0 0 0 0 1 0 0]
```

```
#evaluating the model
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.03211517165005537

Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00         9
           1       0.09      1.00      0.17        29
           2       0.00      0.00      0.00        24
           3       0.00      0.00      0.00        18
           5       0.00      0.00      0.00        90
           6       0.00      0.00      0.00         2
           7       0.00      0.00      0.00        23
           8       0.00      0.00      0.00        10
           9       0.00      0.00      0.00        22
          10       0.00      0.00      0.00       134
          11       0.00      0.00      0.00        13
          12       0.00      0.00      0.00         4
          13       0.00      0.00      0.00       122
          14       0.00      0.00      0.00        43
          15       0.00      0.00      0.00        80
          16       0.00      0.00      0.00         2
          17       0.00      0.00      0.00       276
          18       0.00      0.00      0.00         2

    accuracy                           0.03       903
```

```
   macro avg       0.01      0.06      0.01       903
weighted avg       0.00      0.03      0.01       903


Confusion Matrix:
 [[  0   9   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0  29   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [ 24   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [ 18   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [ 90   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  2   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0  23   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0  10   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0  22   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [123  11   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  7   6   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   4   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [ 90  32   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [ 43   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [ 11  69   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   2   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [175 101   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  2   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]]
```

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd

# Load the data (using your correct file name from page 1)
data = pd.read_csv('Crop and fertilizer dataset.csv')

# Rename columns to their cleaned names (as done on page 1)
data.rename(columns={
    'District_Name': 'District',
    'Soil_color': 'Soil_Color',
    'Nitrogen': 'N',
    'Phosphorus': 'P',
    'Potassium': 'K', # Fixed ' ' to 'K'
    'PH': 'pH', # Fixed pH: PH" to 'pH'
    'Rainfall': 'Rainfall',
    'Temperature': 'Temp',
    'Crop': 'Crop',
    'Fertilizer': 'Fertilizer',
    'Link': 'Reference_Link'
}, inplace=True)

# 1. Encode Features (X)
# Re-apply LabelEncoder to the features you need in X: District, Soil_Color, Crop
# X contains: 'District', 'Soil_Color', 'N', 'P', 'K', 'pH', 'Rainfall', 'Temp', 'Crop' [cite: 488, 490]
for col in ['District', 'Soil_Color', 'Crop']:
    le = LabelEncoder()
    # Replace the original string column with the new encoded integer column
    data[col] = le.fit_transform(data[col])
    # Note: If you want to use inverse_transform later, you should save these 'le' objects.

# 2. Encode Target (y)
# Also encode the target column 'Fertilizer'
le_fert = LabelEncoder()
data['Fertilizer'] = le_fert.fit_transform(data['Fertilizer'])

# 3. Define X and y using the newly encoded integer columns
# X is all columns except 'Fertilizer' and 'Reference_Link'
X = data.drop(['Fertilizer', 'Reference_Link'], axis=1) # All columns are now numeric
# y is the encoded 'Fertilizer' column
y = data['Fertilizer']
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# --- Step 1: Split the Data ---
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# --- Step 2: Feature Scaling (This will now work!) ---
sc = StandardScaler()
X_train_scaled = sc.fit_transform(X_train)
```

```
X_test_scaled = sc.transform(X_test)

# --- Step 3: Build and Train the Model ---
rf = RandomForestClassifier(n_estimators=100, random_state=42)
# Using the scaled features
rf.fit(X_train_scaled, y_train)

# --- Step 4: Predict Test Set Results ---
# Random Forest directly outputs the class label
y_pred = rf.predict(X_test_scaled)

# --- Step 5: Evaluate the Model ---
print("--- Model Evaluation ---")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix: \n", confusion_matrix(y_test, y_pred))
```

```
--- Model Evaluation ---
Accuracy: 0.9424141749723145

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       0.79      0.84      0.81        31
           2       0.88      1.00      0.93        21
           3       1.00      1.00      1.00        13
           4       1.00      1.00      1.00         1
           5       0.98      0.98      0.98        96
           6       1.00      1.00      1.00         3
           7       0.96      0.96      0.96        25
           8       1.00      1.00      1.00        10
           9       0.85      1.00      0.92        22
          10       0.94      0.96      0.95       134
          11       1.00      1.00      1.00        14
          12       1.00      1.00      1.00         5
          13       0.94      0.92      0.93       114
          14       1.00      1.00      1.00        43
          15       0.90      0.87      0.88        83
          16       1.00      1.00      1.00         1
          17       0.97      0.93      0.95       273
          18       0.57      1.00      0.73         4

    accuracy                           0.94       903
   macro avg       0.93      0.97      0.95       903
weighted avg       0.94      0.94      0.94       903


Confusion Matrix:
 [[ 10   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0]
 [  0  26   0   0   0   0   0   0   0   0   0   0   0   0   0   1   0   4
    0]
 [  0   0  21   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0]
 [  0   0   0  13   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0]
 [  0   0   0   0   1   0   0   0   0   0   0   0   0   0   0   0   0   0
    0]
 [  0   0   0   0   0  94   0   0   0   0   0   0   0   0   0   0   0   2
    0]
 [  0   0   0   0   0   0   3   0   0   0   0   0   0   0   0   0   0   0
    0]
 [  0   0   0   0   0   0   0  24   0   0   0   0   0   0   0   0   0   1
    0]
 [  0   0   0   0   0   0   0   0  10   0   0   0   0   0   0   0   0   0
    0]
 [  0   0   0   0   0   0   0   0   0  22   0   0   0   0   0   0   0   0
    0]
 [  0   0   3   0   0   0   0   0   0   0 129   0   0   1   0   0   0   1
    0]
 [  0   0   0   0   0   0   0   0   0   0   0  14   0   0   0   0   0   0
    0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   5   0   0   0   0   0
    0]
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# --- Step 1: Split the Data (Assumes X and y are available and numeric) ---
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# --- Step 2: Feature Scaling ---
sc = StandardScaler()
X_train_scaled = sc.fit_transform(X_train)
X_test_scaled = sc.transform(X_test)

# --- Step 3: Build and Train the Model ---
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train_scaled, y_train)

# --- Step 4: Predict Test Set Results ---
y_pred = rf.predict(X_test_scaled)

# --- FIX: Calculate and save the accuracy to the expected variable ---
test_acc_rf = accuracy_score(y_test, y_pred)

# --- Step 5: Evaluate the Model ---
print("--- Model Evaluation ---")
print("Accuracy:", test_acc_rf) # Use the new variable here
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix: \n", confusion_matrix(y_test, y_pred))

# --- Step 6: Append to Tracking Lists ---
# Ensure the lists exist (if not defined globally)
try:
    model.append('Random Forest')
    acc.append(test_acc_rf)
except NameError:
    # Handle case where model and acc lists haven't been initialized yet
    print("\nWarning: 'model' or 'acc' lists were not initialized. Initializing them now.")
    global model
    global acc
    model = ['Random Forest']
    acc = [test_acc_rf]

# --- Step 7: Print Updated Lists for confirmation ---
print("\n--- Model Tracker Update Confirmation ---")
print("Length of model:", len(model))
print("Length of acc:", len(acc))
print("Models:", model)
print("Accuracies:", acc)
```

```
--- Model Evaluation ---
Accuracy: 0.9424141749723145

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       0.79      0.84      0.81        31
           2       0.88      1.00      0.93        21
           3       1.00      1.00      1.00        13
           4       1.00      1.00      1.00         1
           5       0.98      0.98      0.98        96
           6       1.00      1.00      1.00         3
           7       0.96      0.96      0.96        25
           8       1.00      1.00      1.00        10
           9       0.85      1.00      0.92        22
          10       0.94      0.96      0.95       134
          11       1.00      1.00      1.00        14
          12       1.00      1.00      1.00         5
          13       0.94      0.92      0.93       114
          14       1.00      1.00      1.00        43
          15       0.90      0.87      0.88        83
          16       1.00      1.00      1.00         1
          17       0.97      0.93      0.95       273
          18       0.57      1.00      0.73         4

    accuracy                           0.94       903
   macro avg       0.93      0.97      0.95       903
weighted avg       0.94      0.94      0.94       903


Confusion Matrix:
 [[ 10   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0]
 [  0  26   0   0   0   0   0   0   0   0   0   0   0   0   0   1   0   4
    0]
 [  0   0  21   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0]
 [  0   0   0  13   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0]
 [  0   0   0   0   1   0   0   0   0   0   0   0   0   0   0   0   0   0
    0]
 [  0   0   0   0   0  94   0   0   0   0   0   0   0   0   0   0   0   2
    0]
 [  0   0   0   0   0   0   3   0   0   0   0   0   0   0   0   0   0   0
    0]
 [  0   0   0   0   0   0   0  24   0   0   0   0   0   0   0   0   0   1
    0]
```

```
acc = [] # TEST
model = []
acc1=[] # TRIAN
```

```
 [  0   0   3   0   0   0   0   0   0   0 129   0   0   1   0   0   0   1
```

```
# Decision Tree Classifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report
from sklearn import metrics
from sklearn import tree
import warnings
warnings.filterwarnings('ignore')

ds = DecisionTreeClassifier(criterion="entropy", random_state=2, max_depth=5)
ds.fit(X_train, y_train)

# --- Testing predictions ---
y_pred_test = ds.predict(X_test)
test_acc = metrics.accuracy_score(y_test, y_pred_test)
acc.append(test_acc)

# --- Training predictions ---
y_pred_train = ds.predict(X_train)
train_acc = metrics.accuracy_score(y_train, y_pred_train)
acc1.append(train_acc)

# Store model name
model.append('Decision Tree')

# Print accuracies
print("Decision Tree's Accuracy → Test:", test_acc*100, "Train:", train_acc*100)

# Classification report on TEST predictions only
```

```
print("\nClassification Report (Test Data):\n")
print(classification_report(y_test, y_pred_test))
```

```
Decision Tree's Accuracy → Test: 45.84717607973422 Train: 48.282548476454295

Classification Report (Test Data):

              precision    recall  f1-score   support

           0       0.44      0.40      0.42        10
           1       0.44      1.00      0.61        31
           2       0.35      0.29      0.32        21
           3       0.00      0.00      0.00        13
           4       0.50      1.00      0.67         1
           5       0.55      0.69      0.61        96
           6       0.30      1.00      0.46         3
           7       0.00      0.00      0.00        25
           8       0.29      0.20      0.24        10
           9       0.61      0.91      0.73        22
          10       0.44      0.44      0.44       134
          11       0.38      0.36      0.37        14
          12       1.00      0.40      0.57         5
          13       0.62      0.18      0.27       114
          14       0.00      0.00      0.00        43
          15       0.45      0.47      0.46        83
          16       0.00      0.00      0.00         1
          17       0.43      0.56      0.48       273
          18       0.29      1.00      0.44         4

    accuracy                           0.46       903
   macro avg       0.37      0.47      0.37       903
weighted avg       0.43      0.46      0.42       903
```

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report, accuracy_score

# Train model
NaiveBayes = GaussianNB()
NaiveBayes.fit(X_train, y_train)

# --- Testing predictions ---
y_pred_test = NaiveBayes.predict(X_test)
test_acc = accuracy_score(y_test, y_pred_test)
acc.append(test_acc)

# --- Training predictions ---
y_pred_train = NaiveBayes.predict(X_train)
train_acc = accuracy_score(y_train, y_pred_train)
acc1.append(train_acc)

# Store model name
model.append('Naive Bayes')

# Print accuracies
print("Naive Bayes Accuracy → Test:", test_acc, "Train:", train_acc)

# Classification report (only for TEST data)
print("\nClassification Report (Test Data):\n")
print(classification_report(y_test, y_pred_test))
```

```
Naive Bayes Accuracy → Test: 0.39867109634551495 Train: 0.39501385041551246

Classification Report (Test Data):

              precision    recall  f1-score   support

           0       0.35      0.60      0.44        10
           1       0.44      1.00      0.61        31
           2       0.25      0.19      0.22        21
           3       0.13      1.00      0.23        13
           4       0.00      0.00      0.00         1
           5       0.53      1.00      0.70        96
           6       0.30      1.00      0.46         3
           7       0.39      0.92      0.55        25
           8       0.40      1.00      0.57        10
           9       0.58      1.00      0.73        22
          10       0.39      0.09      0.15       134
          11       0.00      0.00      0.00        14
          12       0.14      0.20      0.17         5
          13       0.44      0.60      0.51       114
```

```
           14       0.41      0.21      0.28        43
           15       0.32      0.34      0.33        83
           16       0.00      0.00      0.00         1
           17       0.42      0.11      0.17       273
           18       0.29      1.00      0.44         4

    accuracy                           0.40       903
   macro avg       0.30      0.54      0.35       903
weighted avg       0.40      0.40      0.33       903
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score

# Train model
LogReg = LogisticRegression(random_state=2, max_iter=1000)  # added max_iter for stability
LogReg.fit(X_train, y_train)

# --- Test predictions ---
y_pred_test = LogReg.predict(X_test)
test_acc = accuracy_score(y_test, y_pred_test)
acc.append(test_acc)

# --- Train predictions ---
y_pred_train = LogReg.predict(X_train)
train_acc = accuracy_score(y_train, y_pred_train)
acc1.append(train_acc)

# Store model name
model.append('Logistic Regression')

# Print accuracies
print("Logistic Regression's Accuracy → Test:", test_acc, "Train:", train_acc)

# Classification report only for TEST set
print("\nClassification Report (Test Data):\n")
print(classification_report(y_test, y_pred_test))
```

```
Logistic Regression's Accuracy → Test: 0.40420819490586934 Train: 0.4116343490304709

Classification Report (Test Data):

              precision    recall  f1-score   support

           0       0.47      0.80      0.59        10
           1       0.00      0.00      0.00        31
           2       0.40      0.19      0.26        21
           3       0.00      0.00      0.00        13
           4       0.00      0.00      0.00         1
           5       0.53      0.61      0.57        96
           6       0.00      0.00      0.00         3
           7       0.69      0.36      0.47        25
           8       0.36      0.40      0.38        10
           9       0.41      0.50      0.45        22
          10       0.31      0.22      0.26       134
          11       0.43      0.21      0.29        14
          12       0.00      0.00      0.00         5
          13       0.39      0.21      0.27       114
          14       0.36      0.63      0.46        43
          15       0.30      0.30      0.30        83
          16       0.00      0.00      0.00         1
          17       0.42      0.59      0.49       273
          18       0.25      0.25      0.25         4

    accuracy                           0.40       903
   macro avg       0.28      0.28      0.27       903
weighted avg       0.38      0.40      0.38       903
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


# Suppress minor warnings for cleaner output
warnings.filterwarnings('ignore')

# --- 1. Data Loading and Initial Setup ---
```

```
# Using the correct filename: 'Crop and fertilizer dataset.csv'
try:
    data = pd.read_csv('Crop and fertilizer dataset.csv')
except FileNotFoundError:
    print("Error: 'Crop and fertilizer dataset.csv' not found. Ensure the file is in the correct directory.")
    exit()

# Clean column names based on your notebook (Page 1)
data.rename(columns={
    'District_Name': 'District',
    'Soil_color': 'Soil_Color',
    'Nitrogen': 'N',
    'Phosphorus': 'P',
    'Potassium': 'K',
    'PH': 'pH',
    'Temperature': 'Temp',
    'Link': 'Reference_Link'
}, inplace=True)

# --- 2. Label Encoding (Categorical to Numeric) ---

# Encode Features: District, Soil_Color, Crop
categorical_features = ['District', 'Soil_Color', 'Crop']
for col in categorical_features:
    le = LabelEncoder()
    # Apply fit_transform to convert strings into integers (0 to N-1)
    data[col] = le.fit_transform(data[col])

# Encode Target: Fertilizer
le_fert = LabelEncoder()
data['Fertilizer'] = le_fert.fit_transform(data['Fertilizer'])

# --- 3. Define Features (X) and Target (y) ---

# X is all numeric columns except 'Fertilizer' and the link column
X = data.drop(['Fertilizer', 'Reference_Link'], axis=1)
y = data['Fertilizer']

# --- 4. Train-Test Split ---
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# --- 5. Feature Scaling (CRUCIAL FOR SVM) ---
# StandardScaler is vital for SVM to work effectively
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# --- 6. Build and Train the SVM Model ---
# Using a linear kernel first (fast and often effective).
# 'C' is a regularization parameter; smaller C means stronger regularization.
print("Training Support Vector Machine (Linear Kernel)...")
svm_model = SVC(kernel='linear', C=1.0, random_state=42)
svm_model.fit(X_train_scaled, y_train)
print("Training complete.")

# --- 7. Prediction ---
y_pred_test = svm_model.predict(X_test_scaled)
y_pred_train = svm_model.predict(X_train_scaled)

# --- 8. Evaluation ---

test_acc = accuracy_score(y_test, y_pred_test)
train_acc = accuracy_score(y_train, y_pred_train)

print("\n--- Support Vector Machine Evaluation ---")
print("SVM (Linear Kernel) Accuracy → Test: {:.2f}% | Train: {:.2f}%".format(test_acc * 100, train_acc * 100))

print("\nClassification Report (Test Data):\n")
print(classification_report(y_test, y_pred_test))

print("\nConfusion Matrix (Test Data):\n")
print(confusion_matrix(y_test, y_pred_test))
```

```
Training Support Vector Machine (Linear Kernel)...
Training complete.
```

```
--- Support Vector Machine Evaluation ---
SVM (Linear Kernel) Accuracy → Test: 40.53% | Train: 44.29%

Classification Report (Test Data):

              precision    recall  f1-score   support

           0       0.33      0.40      0.36        10
           1       0.43      0.10      0.16        31
           2       0.00      0.00      0.00        21
           3       0.41      0.54      0.47        13
           4       0.00      0.00      0.00         1
           5       0.54      1.00      0.70        96
           6       0.00      0.00      0.00         3
           7       0.00      0.00      0.00        25
           8       0.40      1.00      0.57        10
           9       0.39      0.64      0.48        22
          10       0.36      0.22      0.27       134
          11       0.29      0.29      0.29        14
          12       0.00      0.00      0.00         5
          13       0.00      0.00      0.00       114
          14       0.39      0.35      0.37        43
          15       0.34      0.37      0.36        83
          16       0.00      0.00      0.00         1
          17       0.38      0.56      0.46       273
          18       0.25      0.25      0.25         4

    accuracy                           0.41       903
   macro avg       0.24      0.30      0.25       903
weighted avg       0.32      0.41      0.34       903


Confusion Matrix (Test Data):

[[  4   0   0   0   0   0   0   0   0   0   6   0   0   0   0   0   0
    0]
 [  0   3   0   0   0   0   0   0   0   0   0   0   0   0   0  15   0  13
    0]
 [  0   0   0   0   0   0   0   0   0   0  18   0   0   0   3   0   0   0
    0]
 [  0   0   0   7   0   0   0   0   0   0   6   0   0   0   0   0   0   0
    0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   1   0
    0]
 [  0   0   0   0   0  96   0   0   0   0   0   0   0   0   0   0   0   0
    0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   1   0   2
    0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0  25
    0]
 [  0   0   0   0   0   0   0   0  10   0   0   0   0   0   0   0   0   0
    0]
 [  0   0   0   0   0   0   0   0   0  14   0   0   0   0   0   7   0   1
    0]
 [  0   0   3   8   0   0   0   0   5   6  29   0   0   0  20   2   0  61
```

```
from sklearn.model_selection import cross_val_score


score_rf = cross_val_score(rf, X, y, cv=5)
print("Cross-validation score of RF is:", score_rf)

# Decision Tree Classifier (assuming 'ds' is initialized)
score_ds = cross_val_score(ds, X, y, cv=5)
print("Cross-validation score of ds is:", score_ds)

# Logistic Regression (assuming 'LogReg' is initialized)
score_logreg = cross_val_score(LogReg, X, y, cv=5)
print("Cross-validation score of LogReg is:", score_logreg)

# Naive Bayes Classifier (assuming 'NaiveBayes' is initialized)
score_nb = cross_val_score(NaiveBayes, X, y, cv=5)
print("Cross-validation score of NaiveBayes is:", score_nb)
```

```
Cross-validation score of RF is: [0.33222591 0.2414175  0.22259136 0.23614191 0.28381375]
Cross-validation score of ds is: [0.29900332 0.25470653 0.21816168 0.21064302 0.29490022]
Cross-validation score of LogReg is: [0.28682171 0.303433   0.26024363 0.29600887 0.28935698]
Cross-validation score of NaiveBayes is: [0.29900332 0.39202658 0.35880399 0.3481153  0.43348115]
```

```
model.append('Random Forest')
acc.append(test_acc_rf)
```

```
print("Length of model:", len(model))
print("Length of acc:", len(acc))
print("Models:", model)
print("Accuracies:", acc)
```

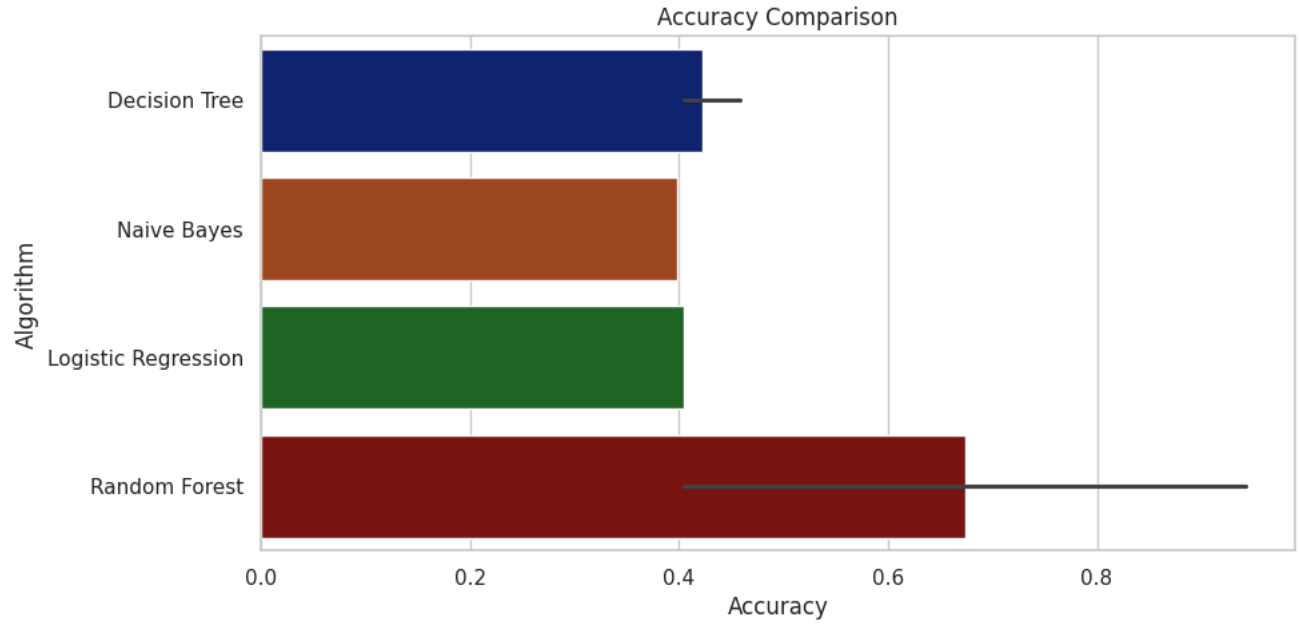
```
Length of model: 7
Length of acc: 6
Models: ['Decision Tree', 'Naive Bayes', 'Logistic Regression', 'Decision Tree', 'Decision Tree', 'Random Forest', 'Random Forest']
Accuracies: [0.4584717607973422, 0.39867109634551495, 0.40420819490586934, 0.4053156146179402, 0.4053156146179402, 0.94241417497231
```

```
min_len = min(len(model), len(acc))
model = model[:min_len]
acc = acc[:min_len]
```

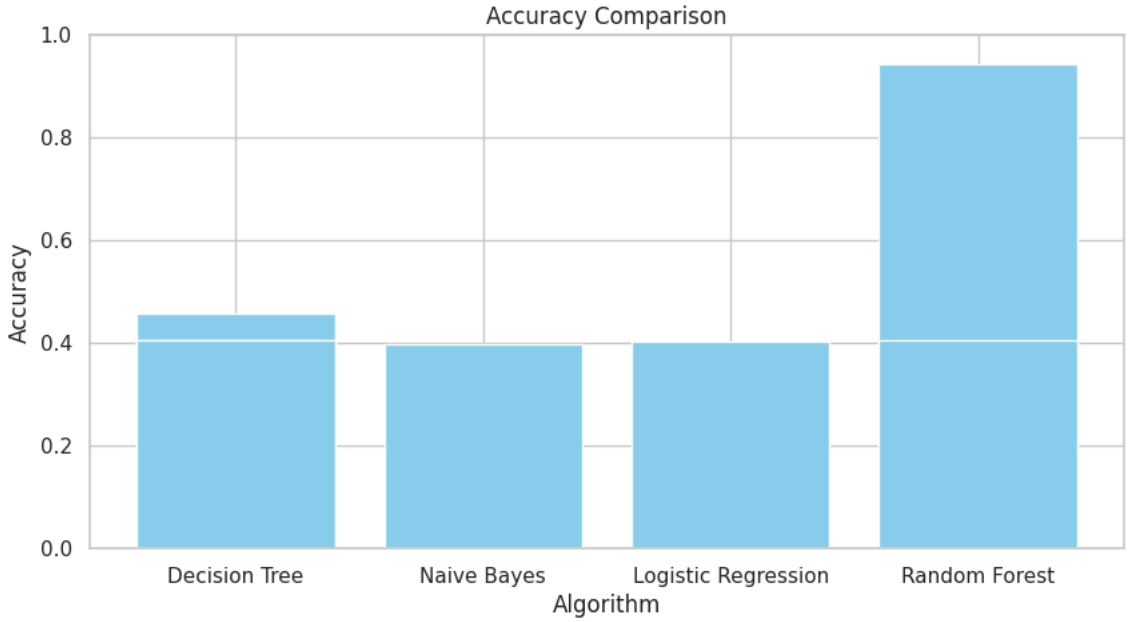
```
print("Length of model:", len(model))
print("Length of acc:", len(acc))
print("Models:", model)
print("Accuracies:", acc)
```

```
Length of model: 7
Length of acc: 7
Models: ['Decision Tree', 'Naive Bayes', 'Logistic Regression', 'Decision Tree', 'Decision Tree', 'Random Forest', 'Random Forest']
Accuracies: [0.4584717607973422, 0.39867109634551495, 0.40420819490586934, 0.4053156146179402, 0.4053156146179402, 0.94241417497231
```

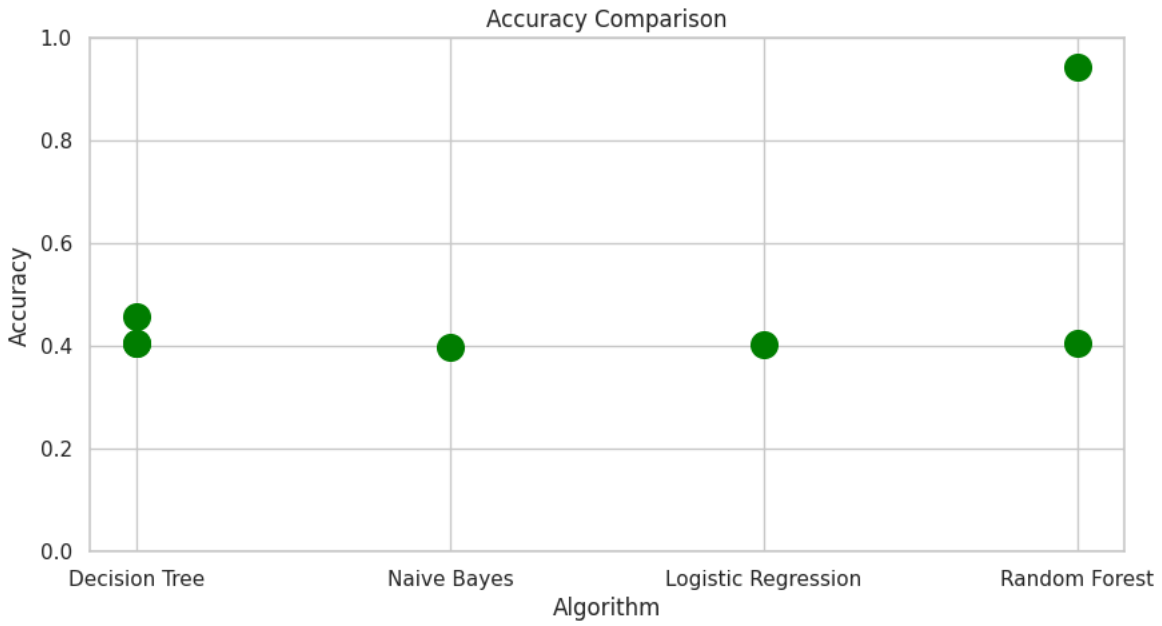
```
# Build dataframe for plotting
results_df = pd.DataFrame({
    "Algorithm": model,
    "Accuracy": acc
})

plt.figure(figsize=(10,5), dpi=100)
sns.barplot(data=results_df, x="Accuracy", y="Algorithm", palette="dark")
plt.title("Accuracy Comparison")
plt.xlabel("Accuracy")
plt.ylabel("Algorithm")
plt.show()
```



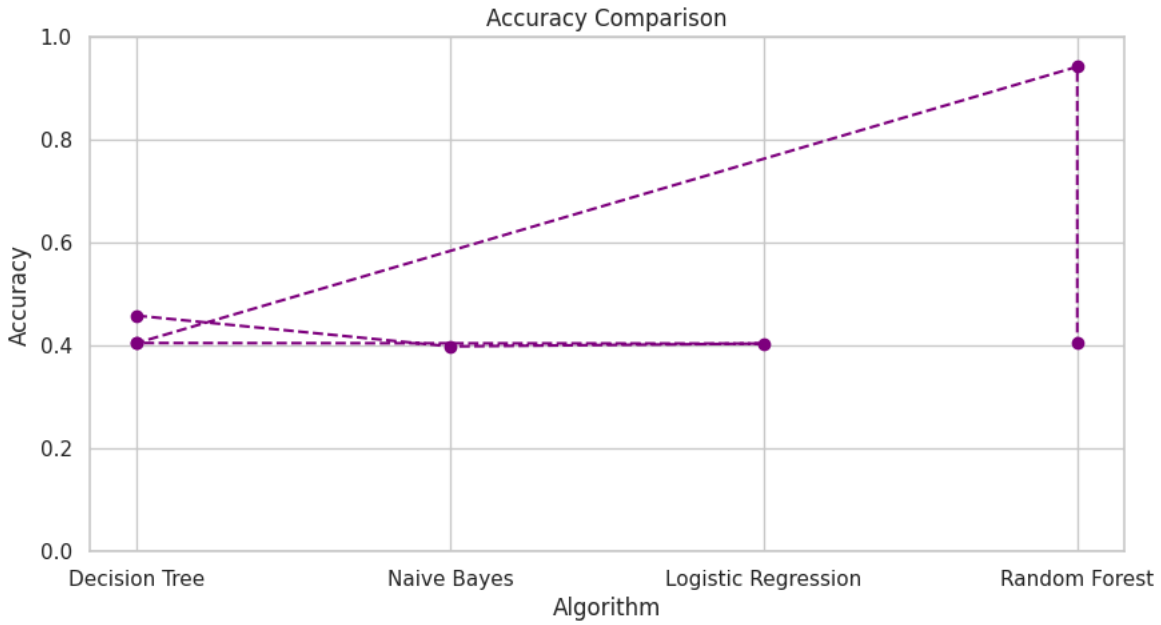
```
plt.figure(figsize=(10,5))
plt.bar(results_df["Algorithm"], results_df["Accuracy"], color='skyblue')
plt.title("Accuracy Comparison")
plt.xlabel("Algorithm")
plt.ylabel("Accuracy")
plt.ylim(0, 1)  # assuming accuracy is between 0 and 1
plt.show()
```




```
plt.figure(figsize=(10,5))
plt.scatter(results_df["Algorithm"], results_df["Accuracy"], s=200, c='green')
plt.title("Accuracy Comparison")
plt.xlabel("Algorithm")
plt.ylabel("Accuracy")
plt.ylim(0, 1)
plt.show()
```




```
plt.figure(figsize=(10,5))
plt.plot(results_df["Algorithm"], results_df["Accuracy"], marker='o', linestyle='--', color='purple')
plt.title("Accuracy Comparison")
plt.xlabel("Algorithm")
plt.ylabel("Accuracy")
plt.ylim(0, 1)
plt.show()
```




```
#pickling the file
import pickle
pickle_out = open('classifier.pkl','wb')
pickle.dump(rf,pickle_out)
pickle_out.close()
```

```
from google.colab import files
files.download("classifier.pkl")
```

```
model = pickle.load(open('classifier.pkl','rb'))
# 9th value (e.g., encoded Crop) is added here
input_9_features = [[34, 67, 62, 0, 1, 7, 0, 30, 5]]
```

```
#pickling the file
import pickle
pickle_out = open('fertilizer.pkl','wb')
pickle.dump(encode_fert,pickle_out)
pickle_out.close()
```

```
fert = pickle.load(open('fertilizer.pkl','rb'))
fert.classes_[1]
```

```
'10:26:26 NPK'
```

```
# testing our model

model = pickle.load(open("classifier.pkl","rb"))
y_pred = model.predict([[34, 67, 62, 0, 1, 7, 0, 30, 5]])

# Convert prediction back to fertilizer name
predicted_fert = fert.inverse_transform(y_pred)
print(predicted_fert[0])
```

```
Urea
```

```
# testing our model

model = pickle.load(open("classifier.pkl","rb"))
y_pred = model.predict([[40, 34, 20, 2, 0, 6, 3, 20, 4]])

# Convert prediction back to fertilizer name
predicted_fert = fert.inverse_transform(y_pred)
print(predicted_fert[0])
```